

Documentación del Taller 3: Programación Discreta

Asignatura: Matemáticas Discretas I
Universidad Nacional de Colombia

Juan Sebastian Rincon Pinzon

Sebastian Polo Alvarez

Bloque A. Criptografía

1. Cifrado César: romper un mensaje antiguo

- **¿Qué problema resuelve el programa?** Permite cifrar y descifrar textos mediante el clásico cifrado por desplazamiento (César), conservando intactos los espacios, los números y los signos de puntuación. Además, incluye una función de ataque por fuerza bruta que evalúa de forma automática los 26 desplazamientos posibles para recuperar mensajes ocultos sin conocer la llave original.
- **¿Qué idea matemática usa?** Se fundamenta en la aritmética modular sobre el alfabeto latino de 26 letras. Cada letra se convierte a un valor numérico según su posición del 0 al 25. El cifrado con llave k se define operando la posición más la llave módulo 26.
 - *¿Por qué el descifrado usa el desplazamiento contrario?* Porque la operación inversa para deshacer el desplazamiento es restar la llave, lo cual equivale matemáticamente a sumar el desplazamiento opuesto, o bien operar con 26 menos la llave.
 - *¿Por qué funciona el ataque de fuerza bruta?* Debido a que el espacio de llaves es extremadamente pequeño, con solo 26 opciones posibles, es computacionalmente trivial probar todas las alternativas en fracciones de segundo y dejar que el ojo humano identifique qué texto tiene sentido semántico.
- **¿Cómo se ejecuta?** Ejecutando el script principal por consola:
`python cifrado_desplazamiento.py`
El programa desplegará un menú interactivo para elegir entre cifrar, descifrar o aplicar fuerza bruta ingresando el texto correspondiente.
- **¿Qué pruebas hicieron?** (Validadas mediante pruebas unitarias en `test_cifrado.py`)
 1. Prueba de texto estándar: Cifrar la cadena exacta "HOLA UNAL" con un desplazamiento de $k=3$, obteniendo correctamente "KROD XQDO".
 2. Prueba de reversibilidad: Verificar que al descifrar un texto previamente cifrado con cualquier llave (ej. "Matematicas Discretas, grupo 4!" con $k=9$) se recupere de forma idéntica la cadena original.
 3. Prueba de integridad de caracteres especiales: Comprobar que símbolos, números y espacios (ej. en "Sala 305, bloque B - 8:15 am") permanezcan completamente

- inalterados durante el desplazamiento.
- 4. Prueba de mayúsculas/minúsculas y rangos equivalentes: Asegurar que las letras minúsculas y mayúsculas no se mezclen y que desplazamientos mayores a 26 o negativos funcionen correctamente mediante congruencia modular.
- 5. Prueba de fuerza bruta: Ingresar un texto cifrado desconocido y verificar que la función devuelva una lista con exactamente 26 candidatos, conteniendo de manera exacta el mensaje original en el índice correspondiente a la llave real.
- **¿Qué limitaciones tiene la solución?** Al estar restringido estrictamente al alfabeto inglés de 26 letras (sin la letra ñ ni acentos), los caracteres especiales con tilde no estándar no se desplazan correctamente y se comportan como símbolos fijos.

2. RSA de juguete: llaves, cifrado y descifrado

- **¿Qué problema resuelve el programa?** Simula el ciclo completo de un criptosistema de llave pública RSA a escala educativa: genera los parámetros de las llaves a partir de dos números primos, calcula el inverso modular y permite cifrar y descifrar mensajes numéricos enteros respetando las congruencias matemáticas.
- **¿Qué idea matemática usa?** Utiliza la aritmética modular avanzada y las propiedades de la función phi de Euler:
 1. Se eligen dos primos distintos p y q , calculando el módulo n como el producto de p por q , y la función phi de n como el producto de $(p - 1)$ por $(q - 1)$.
 2. Se selecciona un exponente público e tal que el máximo común divisor entre e y phi de n sea igual a 1 (coprimos).
 3. Mediante el Algoritmo de Euclides Extendido y la Identidad de Bézout, se halla el inverso multiplicativo modular d tal que el producto de e por d sea congruente con 1 módulo phi de n .
 4. El cifrado y descifrado operan bajo exponenciación modular: el texto cifrado C es igual a M elevado a la e módulo n , y el texto descifrado M es igual a C elevado a la d módulo n .

Papel de los primos y congruencias: Los primos p y q aseguran que la factorización de n sea difícil en sistemas reales (dificultad que sustenta la seguridad de RSA). El inverso modular d actúa como la llave privada que revierte exactamente la transformación hecha por la llave pública e gracias al Pequeño Teorema de Fermat y las propiedades de los anillos de enteros módulo n .

- **¿Cómo se ejecuta?**
`python rsa_juguete.py`
 Se interactúa a través de la consola para registrar los valores de p , q y e , y posteriormente operar el cifrado y descifrado de un mensaje M .
- **¿Qué pruebas hicieron?** (Validadas mediante pruebas unitarias en `test_rsa_juguete.py`)
 - Caso de prueba obligatorio del taller: Usando los parámetros de prueba $p = 61$, $q = 53$ y $e = 17$, el programa computa de forma exacta $n = 3233$, $\phi(n) = 3120$ y el inverso modular privado $d = 2753$. Al cifrar el mensaje $M = 65$, se obtiene exactamente $C = 2790$, y al aplicar el descifrado se recupera de manera íntegra $M = 65$.
 - Pruebas de validación y robustez: Se verificó que el sistema lance errores controlados ante exponentes no coprimos, valores no primos, números iguales para p y q , o intentos de cifrar mensajes mayores o iguales al módulo n .
- **¿Qué limitaciones tiene la solución académica?** Está diseñado exclusivamente para

finés didácticos con números enteros pequeños ingresados por el usuario. No implementa relleno de seguridad ni maneja números primos criptográficamente seguros de cientos de bits, por lo que es vulnerable a ataques de fuerza bruta directos sobre la factorización de n .

3. MPC básico: calcular un promedio sin mostrar los datos

- **¿Qué problema resuelve el programa?** Simula un protocolo de Computación Multipartita Segura (MPC) donde tres servidores independientes procesan notas académicas individuales (entre 0 y 50) fraccionadas en partes aleatorias, logrando calcular la suma total y el promedio global del grupo sin que ningún servidor conozca las notas originales de los estudiantes.
- **¿Qué idea matemática usa?** Se basa en el secreto compartido en aritmética modular con un número primo grande M igual a 1000003. Para ocultar una nota z :
 1. Se generan dos valores aleatorios uniformes s_1 y s_2 módulo M .
 2. El tercer fragmento s_3 se calcula de manera determinista restando s_1 y s_2 a z módulo M .

De esta forma, la suma de las tres partes reconstruye exactamente el valor original módulo M .

Explicación analítica: Tomar s_1 y s_2 al azar significa que cada fragmento individual tiene la misma probabilidad de ser cualquier número, aportando entropía perfecta e información teóricamente nula sobre el valor real z . Ningún servidor individual posee información parcial que revele la nota del estudiante.

- **¿Cómo se ejecuta?**
`python mpc_basico.py`
Se ingresa una lista de notas separadas por comas (ej. 40, 35, 50, 25) para observar la distribución entre servidores y el resultado final reconstruido.
- **¿Qué pruebas hicieron?** (Validadas mediante pruebas unitarias en `test_mpc_basico.py`)
 - Ejemplo obligatorio del taller: Al introducir el vector de notas [40, 35, 50, 25] cuya suma real es 150, el protocolo reparte los fragmentos aleatorios a los 3 servidores, los cuales realizan sumas aisladas de sus partes. Al reconstruir la información final, el sistema obtiene con total exactitud una suma total de 150 y un promedio global de 37.5.
 - Pruebas de seguridad y restricciones: Se comprobó mediante aserciones que ninguna parte individual aislada (s_1 , s_2 o s_3) sea igual al valor original y que se levanten excepciones de error ante listas vacías o notas fuera del rango permitido (0 a 50).
- **¿Qué limitaciones tiene la solución?** Asume una red de servidores honesta pero curiosa, es decir, los servidores siguen las reglas del protocolo y no coluden activamente entre los tres para sumar sus bases de datos y revelar los secretos originales.

Bloque B. Grafos

4. Ruta más corta: una ciudad como grafo (Dijkstra)

- **¿Qué problema resuelve el programa?** Modela una red de transporte de un campus universitario como un grafo ponderado para encontrar la ruta de desplazamiento más

rápida en minutos entre dos puntos específicos de la ciudad.

- **¿Qué idea matemática usa?** Implementa explícitamente el Algoritmo de Dijkstra sobre un grafo conexo ponderado G compuesto por vértices y aristas.
 - *¿Por qué Dijkstra necesita pesos no negativos?* El algoritmo funciona bajo una lógica ávida: una vez que un vértice se marca como confirmado al hallar su distancia mínima definitiva, asume que ningún camino alternativo futuro que pase por otros nodos podrá disminuir esa distancia. Si existieran aristas con pesos negativos, un rodeo posterior podría reducir drásticamente el costo acumulado, invalidando la optimalidad.
 - *¿Qué significa que un camino sea óptimo?* Significa que la sumatoria de los pesos de las aristas que componen la ruta es estrictamente menor o igual a la sumatoria de cualquier otro camino alternativo existente entre los mismos dos vértices.
- **¿Cómo se ejecuta?**
`python ruta_minima.py`
Ejecuta el grafo de prueba predefinido y muestra en consola la distancia y el recorrido detallado para los pares de prueba seleccionados.
- **¿Qué pruebas hicieron?** (Validadas mediante pruebas unitarias en `test_ruta_minima.py`)
 - Estructura de la red del campus: Se validó que el grafo predefinido contenga exactamente **8 vértices** (Biblioteca, Auditorio, Cafetería, Laboratorio, Coliseo, Posgrados, Bloque5, Bloque6) y **12 aristas** debidamente ponderadas.
 - Consulta de ruta óptima: Para el trayecto de Biblioteca a Bloque6, el algoritmo computa de forma exacta un costo mínimo de 15 minutos siguiendo la ruta ["Biblioteca", "Cafetería", "Bloque6"]. De igual manera, se verificó el recorrido de Auditorio a Posgrados con un costo de 19 minutos a través de ["Auditorio", "Laboratorio", "Posgrados"].
 - Casos límite: Se comprobó que la distancia de cualquier vértice hacia sí mismo es cero y que los vértices aislados devuelven correctamente una distancia infinita.
- **¿Qué limitaciones tiene la solución?** El grafo está codificado de manera estática en una función auxiliar y no soporta pesos negativos ni grafos dinámicos en tiempo real sin una reinstanciación de la estructura.

5. Cierre de una estación: medir el impacto en la red

- **¿Qué problema resuelve el programa?** Calcula el impacto estructural que sufre una red de transporte ante el cierre o inhabilitación temporal de un punto clave (por ejemplo, el cierre de la Cafetería), comparando las rutas óptimas de múltiples pares origen-destino antes y después del incidente.
- **¿Qué idea matemática usa?** Utiliza la teoría de conectividad de grafos y la reevaluación del algoritmo de Dijkstra tras la eliminación de un vértice y la supresión de sus aristas incidentes. Mide la variación en la distancia métrica clasificando los resultados en tres categorías discretas: sin cambio, más largo o desconectado.
- **¿Cómo se ejecuta?**
`python impacto_cierre.py`
Genera de forma automática una tabla formateada en consola indicando el impacto del cierre del vértice seleccionado sobre un conjunto de pares de prueba.
- **¿Qué pruebas hicieron?** (Validadas mediante pruebas unitarias en `test_impacto_cierre.py`)
 - Evaluación de cierre clave: Al inhabilitar el vértice "Cafetería" sobre 5 pares de

prueba específicos, el programa comprueba y confirma cuantitativamente aumentos en las rutas (por ejemplo, un incremento de 11 unidades en la ruta entre Biblioteca y Bloque6, y de 7 unidades hacia el Coliseo y Posgrados), además de identificar trayectos estables (sin cambio).

- Casos de desconexión estructural: Se evaluó un grafo lineal de prueba eliminando un nodo intermedio (Punto3), comprobando que el sistema marca correctamente a los pares afectados con el estado de DESCONECTADO (infinito).

¿Qué limitaciones tiene la solución? El análisis se limita a evaluar un único punto de fallo a la vez, sin contemplar fallas simultáneas en cadena de múltiples estaciones o congestión dinámica de tráfico.

Punto 6. Coloreo de grafos: organizar exámenes sin choques

- **¿Qué problema resuelve?** El funciona como grafo, el cual simula que tiene clases las cuales son vértices y los estudiantes en común serían las aristas entre estos. Las clases no pueden tener parciales al mismo tiempo si comparten estudiantes. Modela de una manera que no se intercepten al haber parciales
- **¿Idea matemática?** Es el problema de coloreo de grafos, resuelto aquí con un algoritmo greedy (voraz): para cada vértice, se mira qué colores ya usaron sus vecinos, y se le asigna el menor color disponible que ningún vecino tenga. No garantiza el número cromático mínimo, pero sí garantiza una coloración válida (sin conflictos) en tiempo razonable.
- **¿Cómo se ejecuta?**
python src/grafos/coloreo_grafos.py
python tests/test_coloreo_grafos.py
- **¿Qué pruebas se hicieron?** Verificación de que el grafo tiene al menos 10 vértices, que el coloreo generado es válido (ningún par de vecinos comparte color), y que todos los vértices del grafo reciben un color.
- **¿Limitaciones?** El algoritmo greedy depende del orden en que se recorren los vértices (el orden de inserción del diccionario en Python), así que el número de colores usado puede no ser el mínimo teórico (el número cromático real del grafo).

Bloque C. Álgebra de Boole, Shannon y computación cuántica

Punto 7. Tablas de verdad y circuitos logicos

- **¿Qué problemas resuelve?** Genera tablas de verdad para expresiones booleanas construidas con AND, OR, NOT y XOR (simulado con !=), y permite evaluar esas mismas expresiones en una entrada concreta.
- **¿Idea matemática?** Cada expresión booleana con n variables tiene 2^n combinaciones posibles de entrada. El programa usa `itertools.product([False, True], repeat=n)` para generar todas esas combinaciones (esto es literalmente el producto cartesiano $\{V,F\}^n$) y evalúa la función lógica en cada una. El XOR se simula con != porque, en lógica booleana, $A \text{ XOR } B$ es verdadero exactamente cuando A y B difieren — que es lo mismo que decir "A distinto de B".
- **¿Cómo se ejecuta?**
python src/boole/operadores_logicos.py
python tests/test_operadores_logicos.py

- **¿Qué pruebas se hicieron?** En `test_operadores_logicos.py`: casos puntuales para cada expresión (verdadero y falso), una prueba que recorre las 8 combinaciones de A,B,C y compara contra la definición matemática directa $((A \text{ and } B) \text{ or } (\text{not } C))$, y una prueba de `evaluar_entrada` en un caso concreto.
- **¿Limitaciones?** Tiene 4 variables fijas, no valida que hayan entradas, no implementa otros mecanismos de lógica proposicional, por ejemplo el $\rightarrow$ o $\leftrightarrow$
- **Pregunta:** Una tabla de verdad se relaciona con un circuito lógico, ya que una tabla de verdad representa todas las posibles opciones del circuito lógico, ya que tienen equivalencia sus símbolos.

Punto 8. Tablas de verdad y circuitos logicos

- **¿Qué problema resuelve?** Dada una función booleana definida por sus miniterminos (las combinaciones de entrada donde vale 1), encuentra una expresión simplificada equivalente en forma de suma de productos (OR de términos AND), y verifica que la simplificación sea realmente equivalente a la original.
- **¿Idea matemática?** Implementa el método de Quine-McCluskey: dos términos binarios que difieren en exactamente un bit se pueden combinar en uno solo, reemplazando ese bit por un "-" (que significa "esa variable no importa"). Esto se repite hasta que ningún término se pueda combinar más — los que sobreviven son los implicantes primos. Luego se seleccionan los esenciales (los que son la única cobertura de algún minterm) y, para lo que falte, se aplica una estrategia golosa (greedy) eligiendo siempre el implicante que cubre más minterminos pendientes.
- **¿Cómo se ejecuta?**
`python src/boole/simplificacion_booleana.py`
`python tests/test_simplificacion_booleana.py`
- **¿Qué pruebas se hicieron?** Pruebas unitarias de las funciones auxiliares (`num_a_binario`, `se_combinan`), el caso sugerido del taller (minterminos $\{1,3,5,7\} \rightarrow$ equivalente a "C"), verificación de equivalencia de tablas, casos límite (función siempre falsa / siempre verdadera), y una prueba con `subTest` que recorre varios conjuntos de minterminos distintos (3 y 4 variables) comprobando que la simplificación nunca cambia el comportamiento.
- **¿Limitaciones?** No soporta muchas variables
- **Pregunta:** Un minitermino es una expresión de algebra booleana formada por la multiplicación lógica de todas las variables de la función, en cualquiera de sus opciones, normal o negada.

Punto 9. Shannon, medir la información de un mensaje

- **¿Qué problema resuelve?** Mide cuánta información (o incertidumbre) contiene un mensaje. Un mensaje que siempre dice lo mismo no te sorprende, así que aporta poca información nueva. Un mensaje muy variado, donde cualquier símbolo puede aparecer con probabilidad similar, es más impredecible y por lo tanto contiene más información.

- **¿Idea matemática?**

$$H = -\sum p_i \log_2(p_i)$$

Donde p_i es la cantidad de veces que aparece el símbolo sobre la cantidad total de símbolos

- **¿Cómo se ejecuta?**

```
python src/cuantica/shannon.py
python tests/test_shannon.py
```

- **¿Qué pruebas se hicieron?**
 - Verificación de conteo: que frecuencia_texto cuente bien las repeticiones y no devuelva None (esto atrapó el bug original del return faltante)
 - Casos analíticos exactos: textos donde se puede calcular la entropía a mano y compararla con lo que da el código (ej. "AABB" debe dar exactamente $H=1.0$, "ABCD" debe dar exactamente $H=2.0$).
 - Condición central del taller: un texto repetitivo debe tener menor entropía que uno variado — se probó directamente comparando los dos valores.
 - Propiedades generales con subTest: para varios textos distintos, la entropía nunca debe ser negativa y nunca debe superar $\log_2(n)$ (n = cantidad de símbolos distintos), que es la cota teórica máxima.
- **¿Limitaciones?** El cálculo trata cada símbolo como independiente de los demás: no captura que en español, por ejemplo, después de una "q" casi siempre viene una "u". Un modelo de entropía más realista consideraría la probabilidad condicional entre símbolos consecutivos (entropía de orden superior), no solo la frecuencia individual.
- **Pregunta:** Primeramente porque la longitud de texto sería solo contar los caracteres, además de que calcula la incertidumbre individualmente por la manera de cómo se generan los datos, p_i el cual expresado de otra manera podría ser un porcentual de aparición de un caracter, y este es utilizado en general

Punto 10. Shannon, medir la información de un mensaje

- **¿Qué problema resuelve?** Modela el comportamiento básico de un qubit, la unidad de información de la computación cuántica, sin necesitar hardware cuántico real. Un bit clásico solo puede valer 0 o 1; un qubit puede estar en una superposición de ambos estados a la vez, y solo al "medirlo" colapsa a uno de los dos. El ejercicio busca que entiendas cómo las compuertas cuánticas transforman ese estado y cómo se relacionan las probabilidades matemáticas con lo que observarías si hicieras muchas mediciones
- **¿Idea matemática?** Un qubit se representa como un vector de 2 entradas (complejas):
$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = (\alpha, \beta)$$
Las compuertas cuánticas son **matrices** que actúan sobre ese vector mediante multiplicación matriz-vector
- **¿Cómo se ejecuta?**

```
python src\cuantica\simulador_cuantico.py
python tests\test_qubits.py
```
- **¿Limitaciones?**
 - No hay colapso real del estado: en un computador cuántico de verdad, medir destruye la superposición, el qubit queda fijo en 0 o 1 y ya no puedes "volver a preguntarle" sus probabilidades. Aquí, calcular_probabilidades se puede llamar cuantas veces se quiera sobre el mismo estado sin que nada cambie; solo estamos generando números aleatorios acordes a esas probabilidades, no simulando el colapso físico.
 - Solo funciona con un qubit
 - Las probabilidades observadas nunca serán exactamente iguales a las teóricas: con 1000 mediciones vas a ver cosas como 0.507/0.493 en vez de 0.5/0.5 exacto eso es esperado (ley de los grandes números), no un error del código.
- **Pregunta:** En un computador cuántico real, medir un qubit es un evento físico irreversible:

el estado colapsa. Antes de medir, el qubit puede estar en superposición, $(\alpha|0\rangle + \beta|1\rangle)$; en el instante en que se mide, dejas de tener esa superposición y el qubit queda fijo en 0 o en 1. Si quisieras "volver a medir" para ver si obtienes otro resultado, no puedes, tendrías que preparar el estado de nuevo desde cero. Por eso, para estudiar la distribución de probabilidades reales, hay que repetir todo el experimento (preparar el estado, aplicar las compuertas, medir) muchas veces, no medir muchas veces el mismo qubit.